



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Evan Matlock
August 14th 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
 - Data Collection through API
 - Data Collection with Web Scraping
 - Data Wrangling
 - Exploratory Data Analysis with SQL
 - Exploratory Data Analysis with Data Visualization
 - Interactive Visual Analytics with Folium
 - Machine Learning Prediction
- Summary of all results
 - Exploratory Data Analysis result
 - Interactive analytics in screenshots
 - Predictive Analytics result

Introduction

Falcon 9 rockets are advertised to launch with a cost of 62 million dollars while other competitors can cost 165 million or greater. This is due to recycling of the first stage, thus: if we can determine the result of the first stage, we can likely determine the cost of a launch. This data can be used to educate guess' on SpaceX launch results and cost. The goal is to create a ML pipeline to determine if the first stage will land successfully.

What are we trying to figure out with this data?

- What factors determine success in landing?
- How do these factors interact with each other?
- What needs to happen to guarantee launch success?

Section 1

Methodology

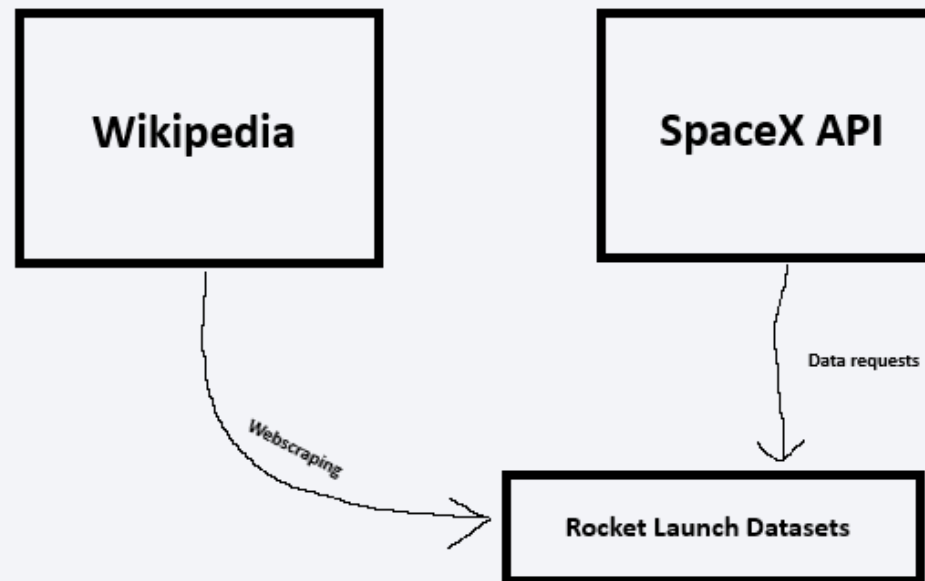
Methodology

Executive Summary

- Data collection methodology:
 - Data was obtained via 2 means:
 - From the SpaceXAPI(<https://api.spacexdata.com/v4/rockets/>)
 - WebScraping(https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches)
- Perform data wrangling
 - Collected data was enriched by creating a landing outcome label based on outcome data after summarizing and analyzing features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Data collected up to this step were normalized, divided in training (and test data sets) to be evaluated by four different classification models, with accuracy of each model being determined from of a variety of factors.

Data Collection

Datasets were collected from the SpaceX API (<https://api.spacexdata.com/v4/rockets/>) and from Wikipedia (https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches) below is a basic flowchart:



Data Collection – SpaceX API

SpaceX offers a public API from where data can be obtained and then used; this API was used according to the flowchart beside and then data is persisted.

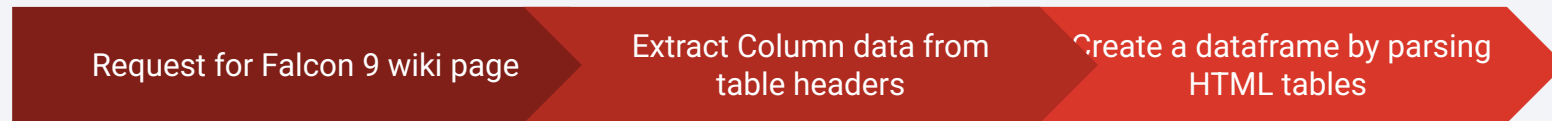
Github relevant notebook:

<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>



Data Collection - Scraping

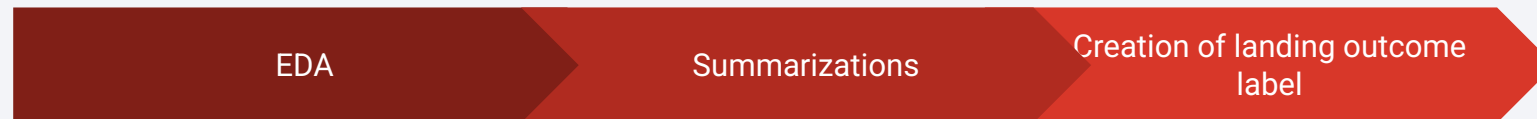
Data from SpaceX launches can also be obtained from Wikipedia. In other words: Data is downloaded from Wikipedia according to the flowchart and made into a dataframe.



Github relevant notebook:
<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/jupyter-labs-webscraping.ipynb>

Data Wrangling

Initially some Exploratory Data Analysis (EDA) was performed on the dataset. Then the summary launches per site, occurrences of each orbit and occurrences of mission outcome per orbit type were calculated. Finally, the landing outcome label was created from the Outcome column.



Github relevant notebook:

<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

EDA with Data Visualization

Below are the charts that were made and how they relate to the findings:

- A flight number vs payload mass (to find that increase in flight number leads to higher success on the first stage).
- A flight number vs launch site (showing that every increase in flight number increased success odds across the board regardless of site)
- A payload mass vs launch site (showing that every site has particular payload sizes that are far more consistent in their success range)
- An orbit vs success rate (showing the success rates of each orbit)
- A flight number vs orbit type (shows that some other factors outside of flight number seem to be affecting the success rate of flights)
- Payload mass vs orbit type (heavy payloads show far more success for Polar, LEO, and ISS, while GTO is more difficult to distinguish)
- Success rate vs year (shows that success overall has been on the rise in the last few years [2013-2019])

Github relevant notebook:

<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/edadataviz.ipynb>

EDA with SQL

Below are the relevant SQL queries performed:

- Displaying unique launch sites
- Displaying 5 records with launch sites beginning with “CCA”
- Displaying total payload mass carried by boosters launched by NASA (CRS)
- Displaying average payload mass carried by booster version F9 v1.1
- Display the date of the first successful ground pad landing
- Display the names of boosters which succeeded in a drone ship landing and have a payload mass greater than 4000 but less than 6000
- Display the total number of successful and failed mission outcomes
- Display the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function
- List the records that will display the month names, failure landing outcomes for drone ships, booster versions, launch_site for the months in year 2015
- Rank the count of landing outcomes between the date of 2010-06-04 and 2017-03-20, in descending order

Github relevant notebook:

<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/jupyter-labs-sql-sqlite.ipynb>

Build an Interactive Map with Folium

The map made with folium has several objects to help clarify what one would be looking at. Below we have descriptions and what they're for:

- Labels the names(these are for the launch sites)
- Circles (these are also for the launch sites)
- Markers if you zoom into the circles (these are for indicating successful and failed launches)
- Lines near one of the sites(these within the context of this map are used for determining distance from certain things)

Github relevant link:

https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

Several elements were added to make this application useful such as: a drop-down so you can select a specific site to focus on, a pie chart based on that choice to show the success rate, a range slider to show the difference for payload mass, and a callback function to help create a scatter chart.

Github relevant link:

<https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/spacex-dash-app.py>

Predictive Analysis (Classification)

This was done through loading data using both numpy and pandas to split the data for training and testing. We used various ML models and tuned parameters using accuracy as the metric for the models and found the best performing classification model.

Github relevant link:

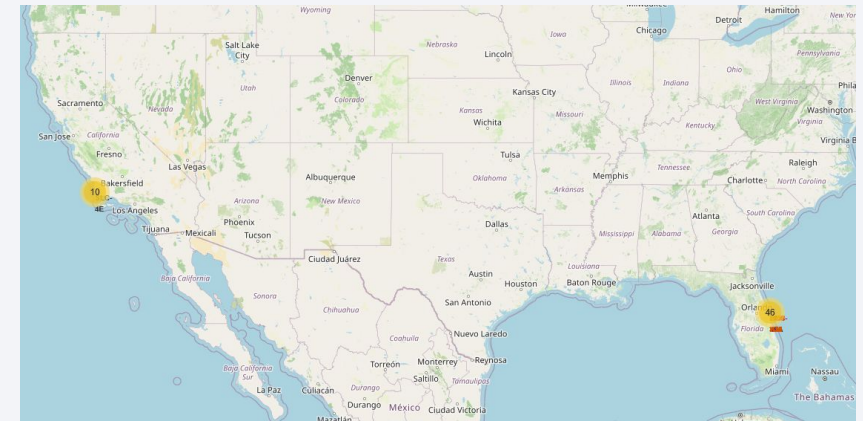
https://github.com/erkam002/Applied-Data-Science-Capstone-Files/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

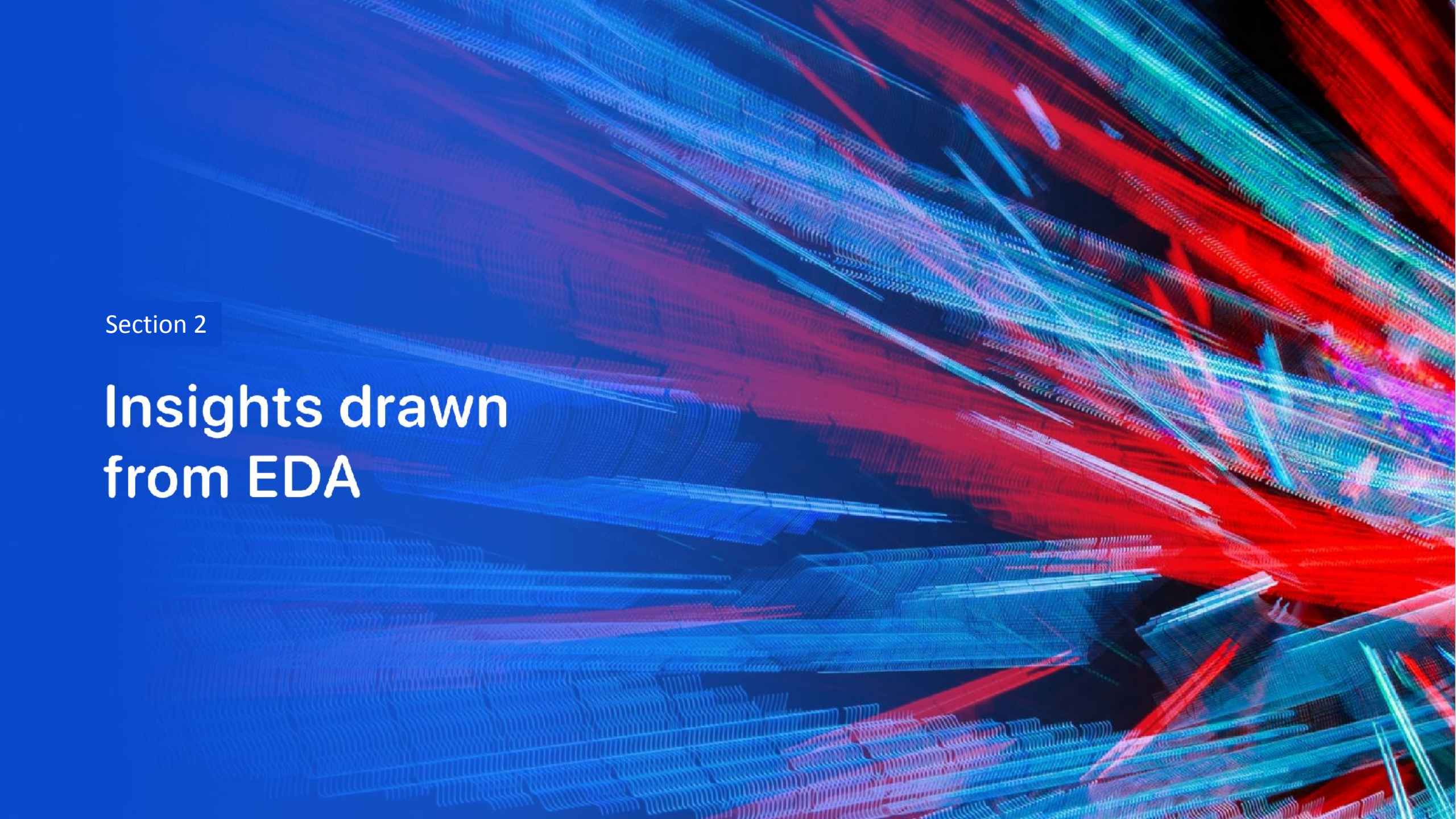
Results

Exploratory data analysis results:

- SpaceX uses 4 different launch sites
- The first launches were done by SpaceX itself and NASA
- The average payload of F9 v1.1 booster is 2,928 kg
- The first successful landing outcome happened in 2015, five years after the first launch
- Many Falcon 9 booster versions were successful at landing in drone ships having payload above the average
- Almost 100% of mission outcomes were successful
- Two booster versions failed at landing in drone ships in 2015: F9 v1.1 B1012 and F9 v1.1 B1015
- The number of landing outcomes drastically improved as years passed.

The sites were mainly in safe areas near the ocean with good infrastructure (such as train tracks) nearby. Majority of launches were on the east coast.



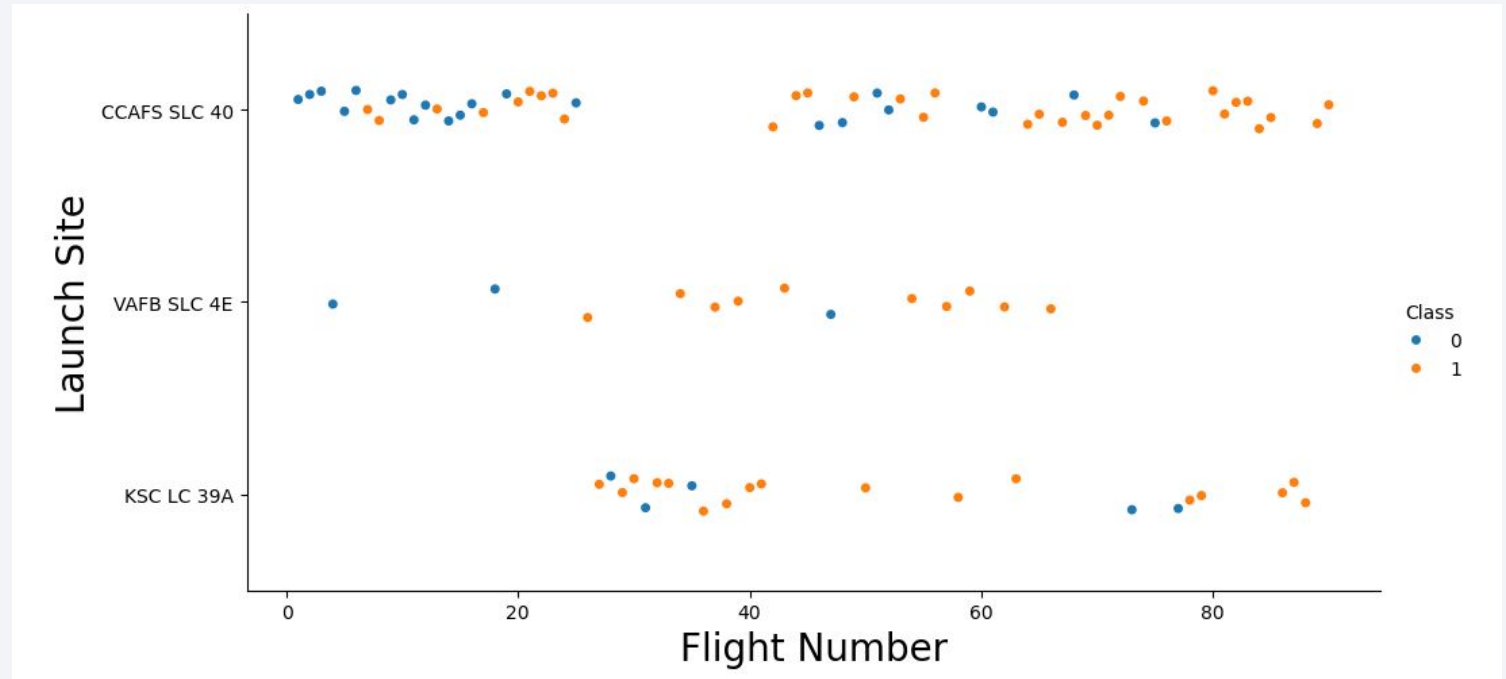
The background of the slide is an abstract composition. It features a solid blue area on the left side, which transitions into a dynamic pattern of diagonal streaks in shades of blue and red on the right. Overlaid on these streaks is a faint, light blue grid pattern, giving the impression of a digital or data-driven environment.

Section 2

Insights drawn from EDA

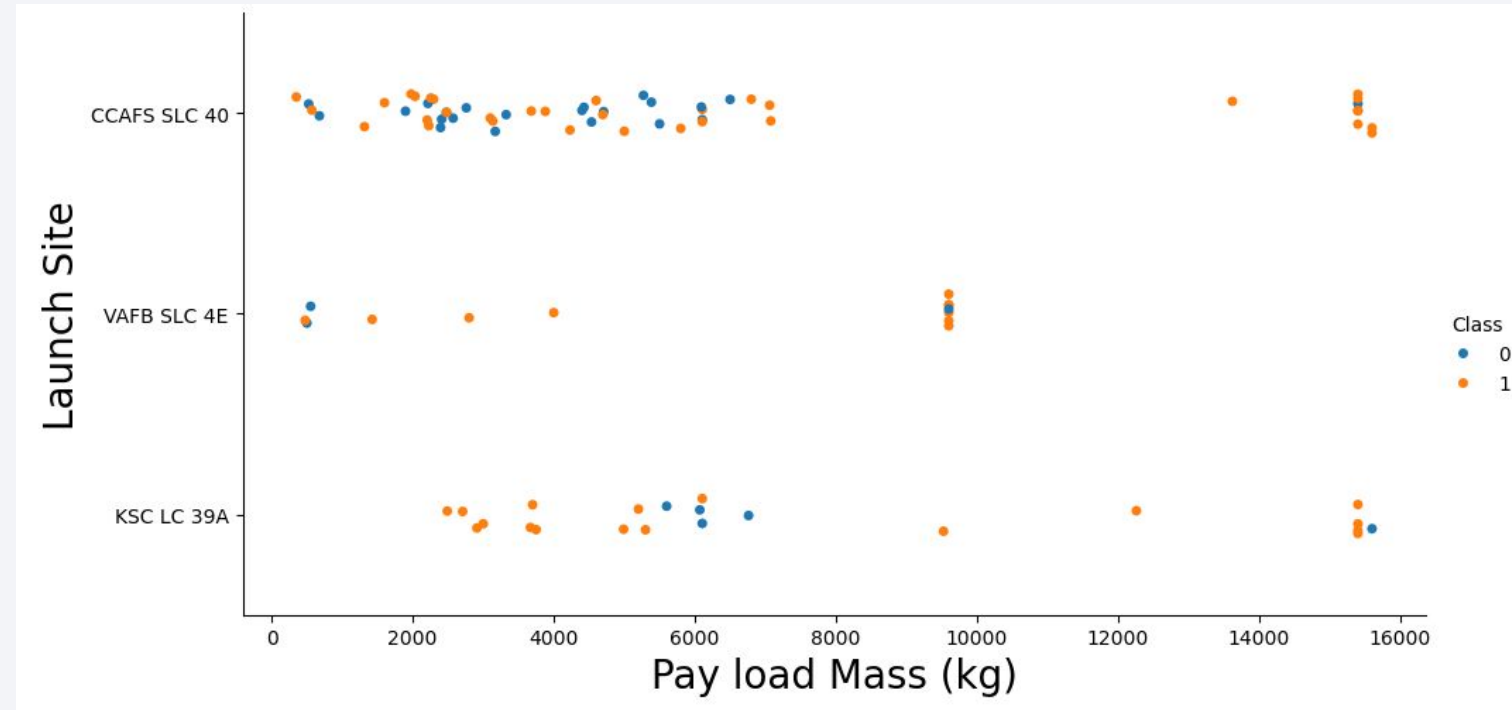
Flight Number vs. Launch Site

The trend on the graph shows as that as the flight number increases the yellow dots (successes) becomes far more consistent across the board.



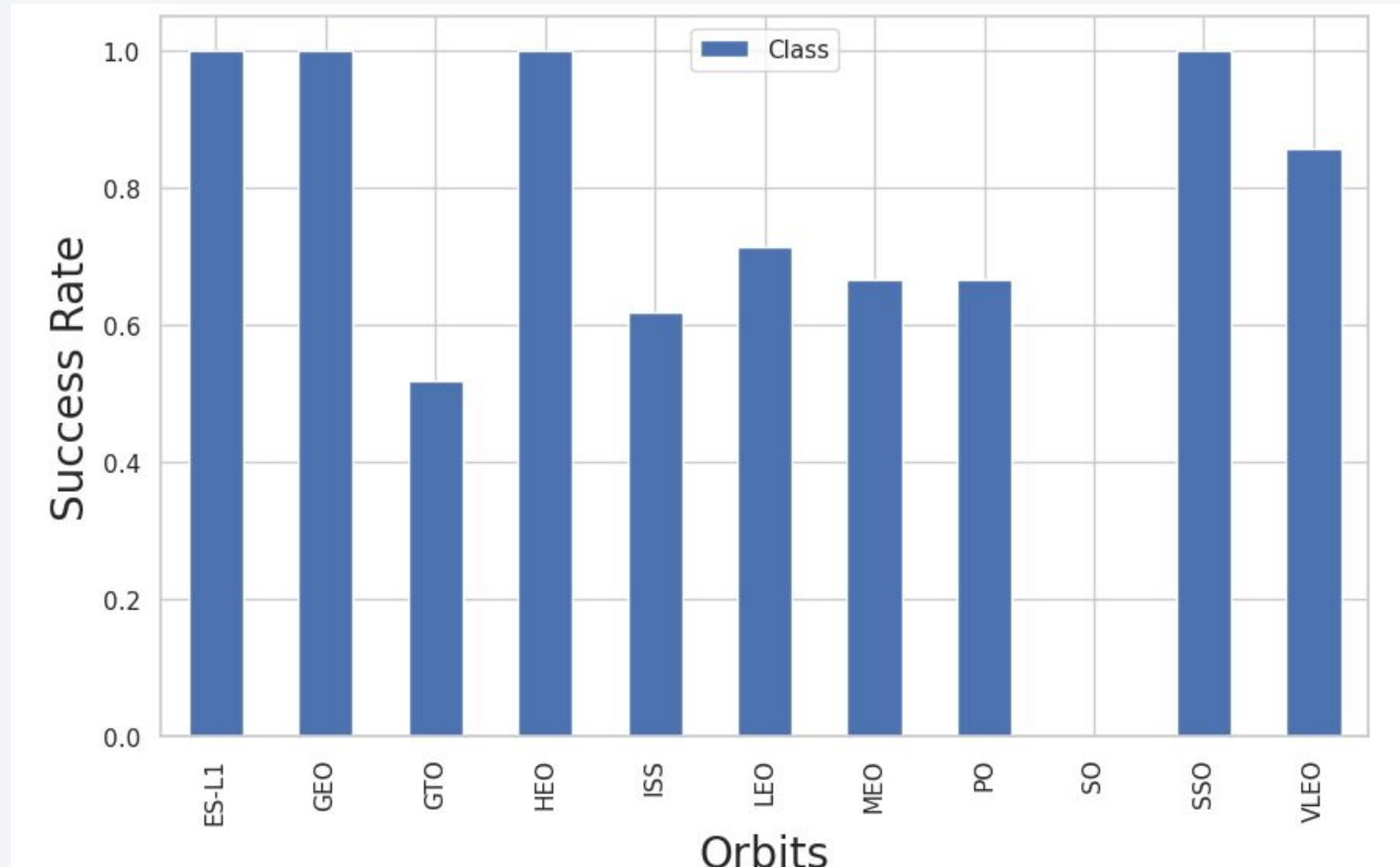
Payload vs. Launch Site

There are a few particular ranges that show the greatest success range for their site, but the most evident example is for CCAFS SLC 40. Showing to be very inconsistent on the lower the flight number but the success rate explodes past 12,000.



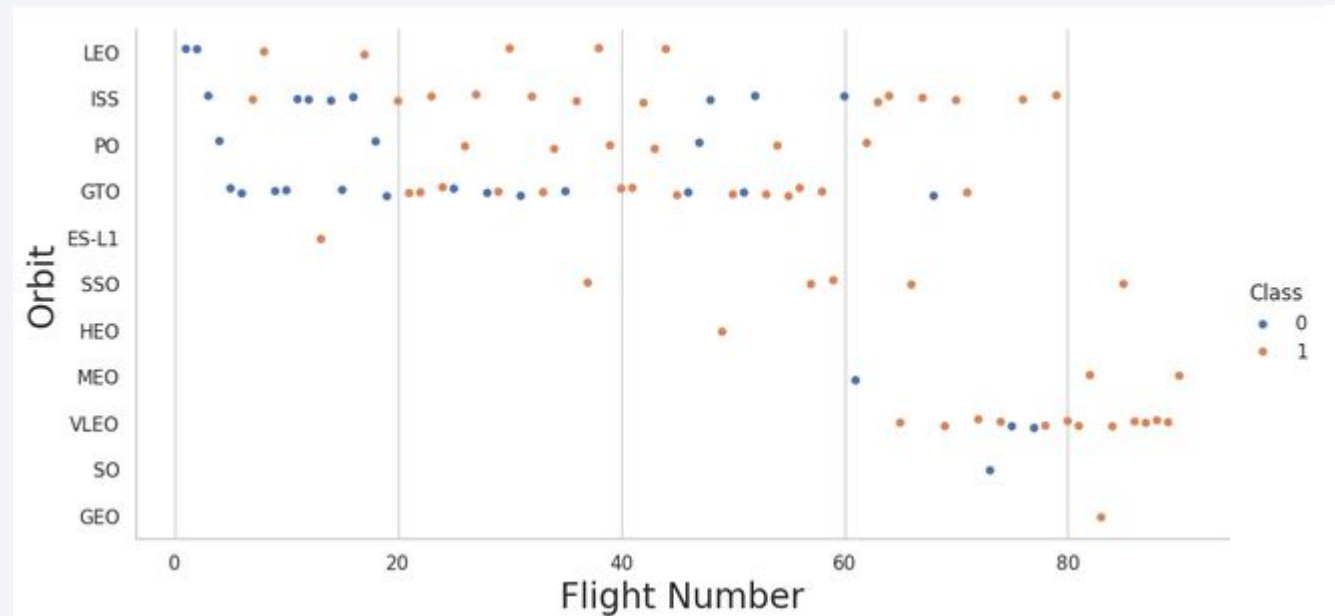
Success Rate vs. Orbit Type

Here we can see the various success rates for launches based on their orbits. We can observe that ES-L1, GEO, HEO, SSO, and VLEO held the highest success rate.



Flight Number vs. Orbit Type

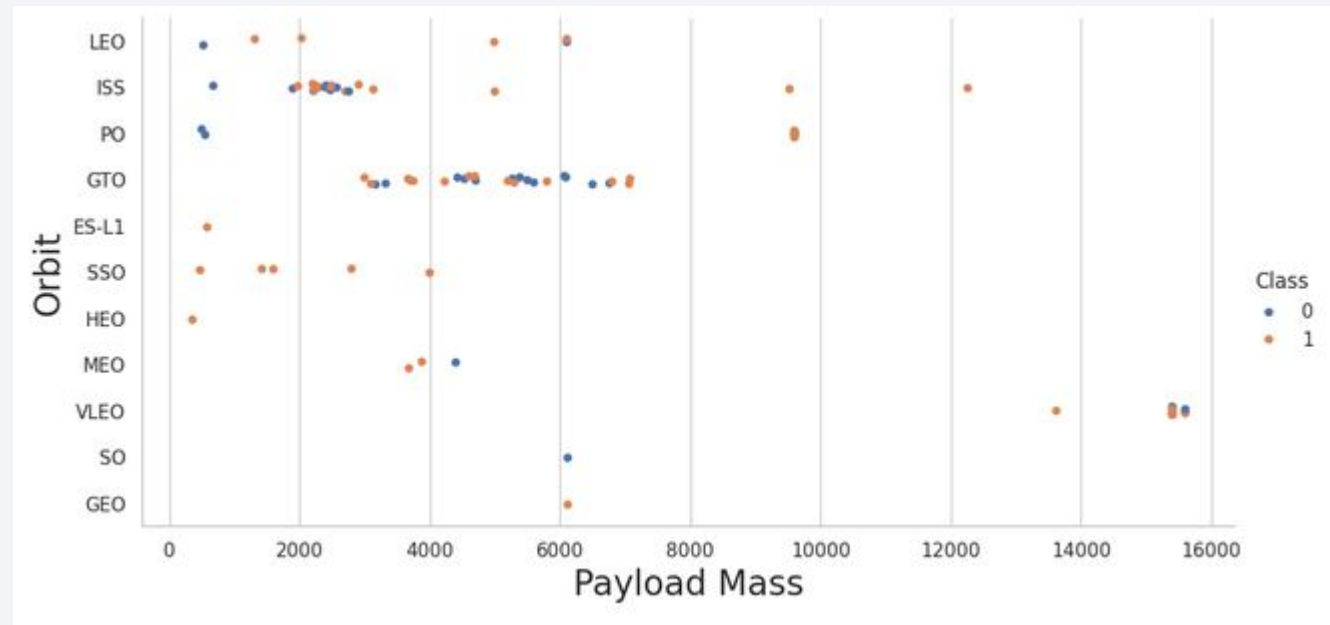
Here we can find a chart depicting orbit types vs the number of flight's comparing the successes and failures. From this we can see that VLEO for instance is consistently very successful from it's high flight number count.



Payload vs. Orbit Type

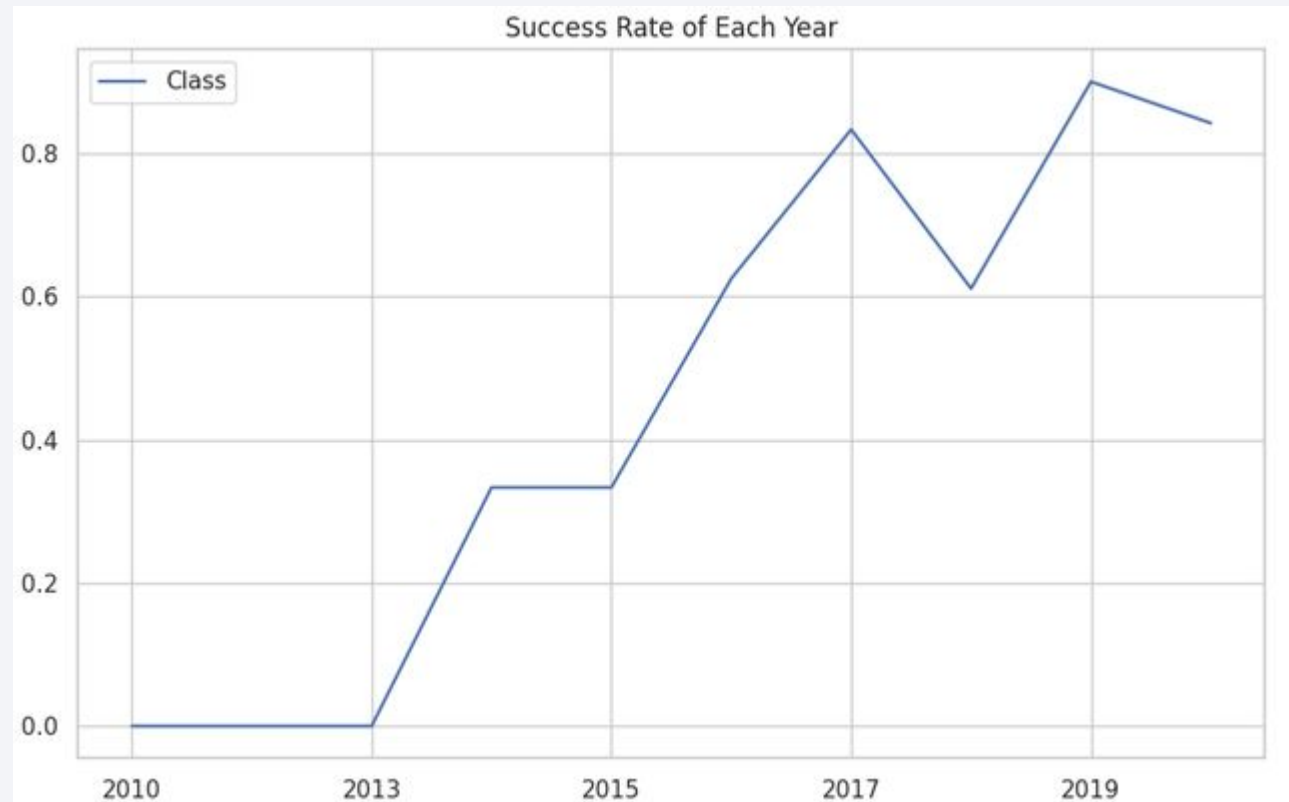
With heavy payloads the successful landing or positive landing rates are more for Polar, LEO, and ISS.

However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.



Launch Success Yearly Trend

Here we can clearly see that as the years go by, the success rate continually climbs higher.



All Launch Site Names

Here we can find the SQL query used to get the individual launch site names. Distinct is used here to make sure the values are the sole instances of that string of text being pulled from the column “LAUNCH_SITE” pulled from the table “SPACEXTBL”.

```
%sql select distinct LAUNCH_SITE from SPACEXTBL;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[34]:
```

Launch_Site

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

Launch Site Names Begin with 'CCA'

Below we can find the query used to obtain launch site names beginning with “CCA”, this is done by first selecting from the proper table, then adding a clarifier being “where Launch_Site like ‘CCA%’ LIMIT 5;”

like ‘CCA%’ is saying everything with CCA at the beginning of the text, where the percent sign is just covering for any text past CCA.

LIMIT 5 is simply limiting the amount of launch sites shown.

```
%sql select * from SPACEXTABLE where Launch_Site like 'CCA%' LIMIT 5;
```

```
* sqlite:///my_data1.db  
one.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

45,596kg is the sum of the payloads. This is calculated using the sum function, pulling from the “PAYLOAD_MASS__KG_” column, and using an additional qualifier with “where (Customer = ‘NASA (CRS)’);” keeps it narrowed down for NASA CRS related rockets.

```
%sql select sum(PAYLOAD_MASS__KG_) as nasa_load_mass from SPACEXTABLE where (Customer = 'NASA (CRS)');
```

```
* sqlite:///my_data1.db
```

```
Done.
```

<u>nasa_load_mass</u>

45596

Average Payload Mass by F9 v1.1

The average is calculated by using the avg function, selecting the PAYLOAD_MASS__KG_ column, and using the additional qualifier of “where (Booster_Version like ‘F9 v1.1%’)” to get: 2534.6666666666665

```
%sql select avg(PAYLOAD_MASS__KG_) as avg_F9v1_load from SPACE_TABLE where (Booster_Version like 'F9 v1.1%');
```

```
* sqlite:///my_data1.db
```

```
Done.
```

<u>avg_F9v1_load</u>

2534.6666666666665

First Successful Ground Landing Date

The first successful ground landing was found using the min() function on the Date column with the additional qualifier of “where Landing_Outcome = ‘Success (ground pad)’

```
%sql select min(Date) from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

min(Date)

2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

This was done by using several qualifiers in “where Landing_Outcome = 'Success (drone ship)' and PAYLOAD_MASS__KG_ > 4000 and PAYLOAD_MASS__KG_ < 6000”

```
%sql select distinct Booster_Version from SPACEXTABLE where Landing_Outcome = 'Success (drone ship)' and PAYLOAD_MASS__KG_ > 4000 and PAYLOAD_MASS__KG_ < 6000;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

I really wanted this piece of code 1 line, so I utilized the count function as well as nesting case within it. This would count the number of successes and put it into one column then stop the case and the same would occur for failures. Giving the results of 100 successes and 1 failure found.

```
%sql select count(case when Mission_Outcome like 'Success%' then 1 end) as Success, count(case when Mission_Outcome like 'Failure%' then 1 end) as Failure from SPACEXTABLE;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Success	Failure
---------	---------

100	1
-----	---

Boosters Carried Maximum Payload

Booster versions which carried the maximum payload were found by selecting both the Booster_Version and the PAYLOAD_MASS__KG_ columns with the qualifier of “PAYLOAD_MASS__KG_ = (select max(Payload_Mass__KG_) from SPACEXTABLE)”

This resulted in the list below:

```
%sql select Booster_Version, PAYLOAD_MASS__KG_ from SPACEXTABLE where PAYLOAD_MASS__KG_ = (select max(Payload_Mass__KG_) from SPACEXTABLE)
```

```
* sqlite:///my_data1.db  
Done.
```

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

Here we looking for launches in a specific set of time in 2015, monthnames doesn't work on SQLite so the course advised substr to substitute as it pulls string from a set position to another set position. So I used that function twice for both the month and the year, as well as selecting the outcome, booster version, and the launch site along with an additional qualifier in the "like 'Failure (drone ship)' and substr(Date,1,4)='2015'"

The additional substr at the end was to have the years appear as numbers, because if it wasn't there it would be shown as "true" where it would show 1 for the year instead.

```
%sql select substr(Date,6,2) as 'month', substr(Date,1,4) as 'year', Landing_Outcome, Booster_Version, Launch_Site from SPACEXTABLE where Landing_Outcome like 'Failure (drone ship)' and substr(Date,1,4)='2015';
```

* sqlite:///my_data1.db
Done.

month	year	Landing_Outcome	Booster_Version	Launch_Site
01	2015	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	2015	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

This is grouping the landing outcomes by number in descending order from 2010-06-04 to 2017-03-20. This was done by selecting the landing outcome column, using the count() function on the landing outcome column and having the additional qualifier specifying to select where the Date column is between 2010-06-04 to 2017-03-20 which would then be grouped by the outcome in a descending count.

```
%sql select Landing_Outcome, count(Landing_Outcome) from SPACEXTABLE where Date between '2010-06-04' and '2017-03-20' group by Landing_Outcome order by count(Landing_Outcome) desc;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Landing_Outcome	count(Landing_Outcome)
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing a curved horizon and a dense network of city lights and clouds. The image is used as a background for the title slide.

Section 3

Launch Sites Proximities Analysis

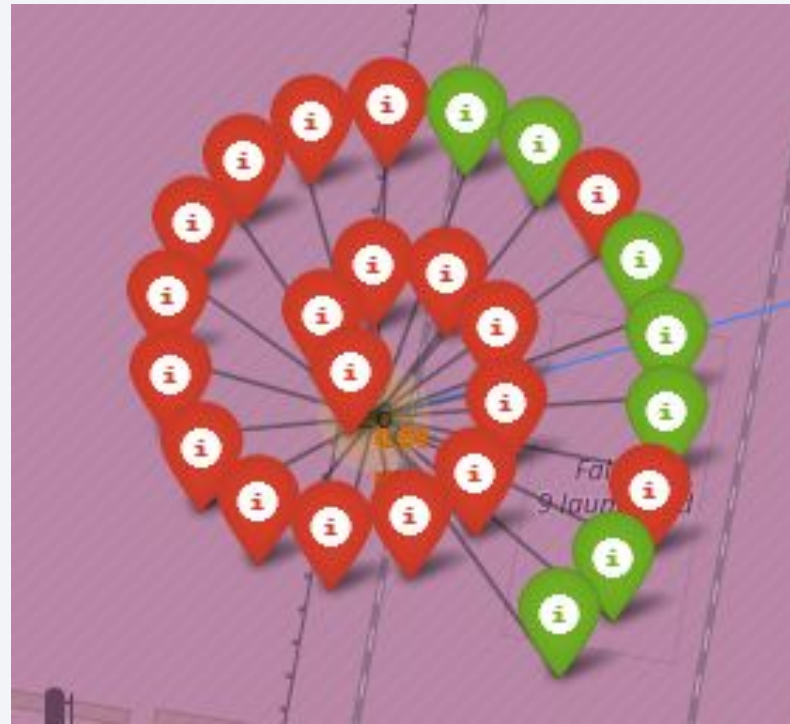
Folium USA Launch Map

This is the broader folium map. You can see a few things here, namely: numbers (the # of launches within the radius), names (launch sites, such as SLC-4E, etc.), a circle showing broadly where the launch sites are, and if you zoom in: green or red markers showing successful or failed launches



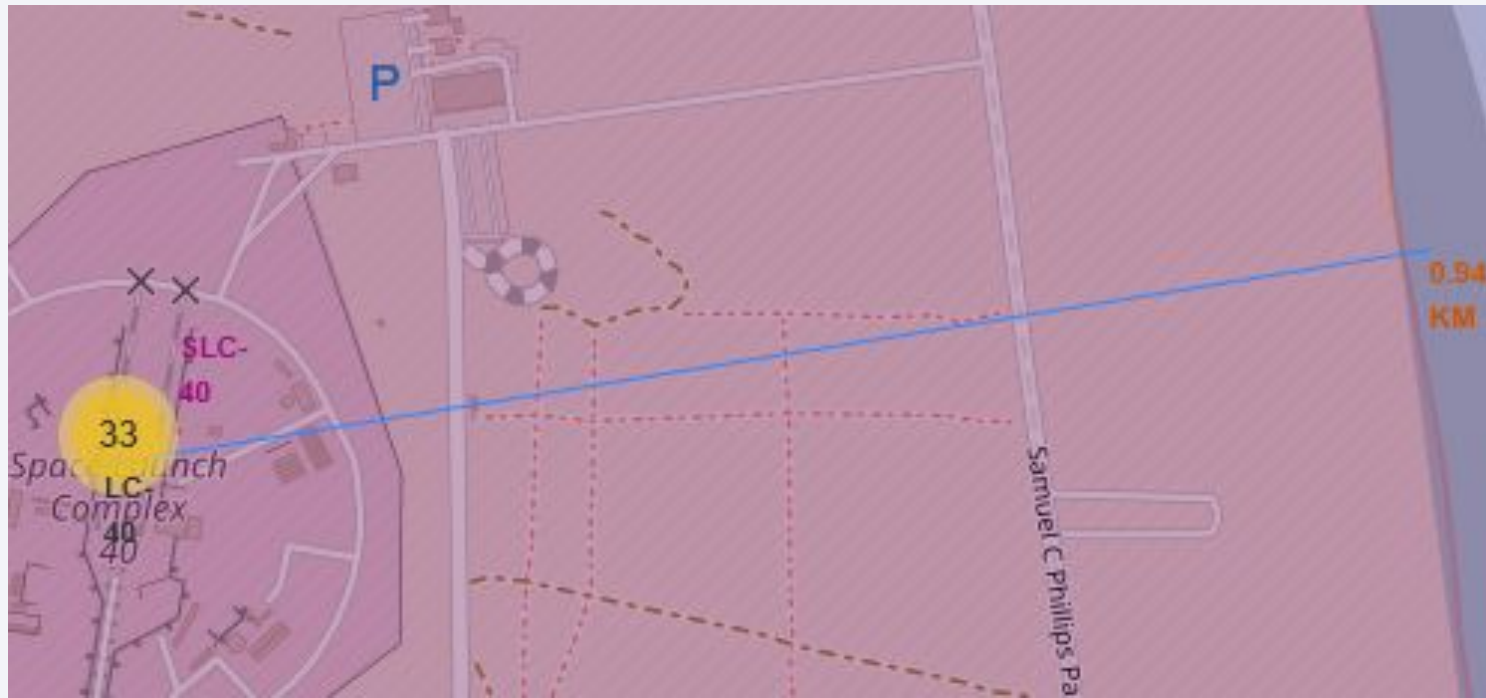
Folium Success/Failure Markers

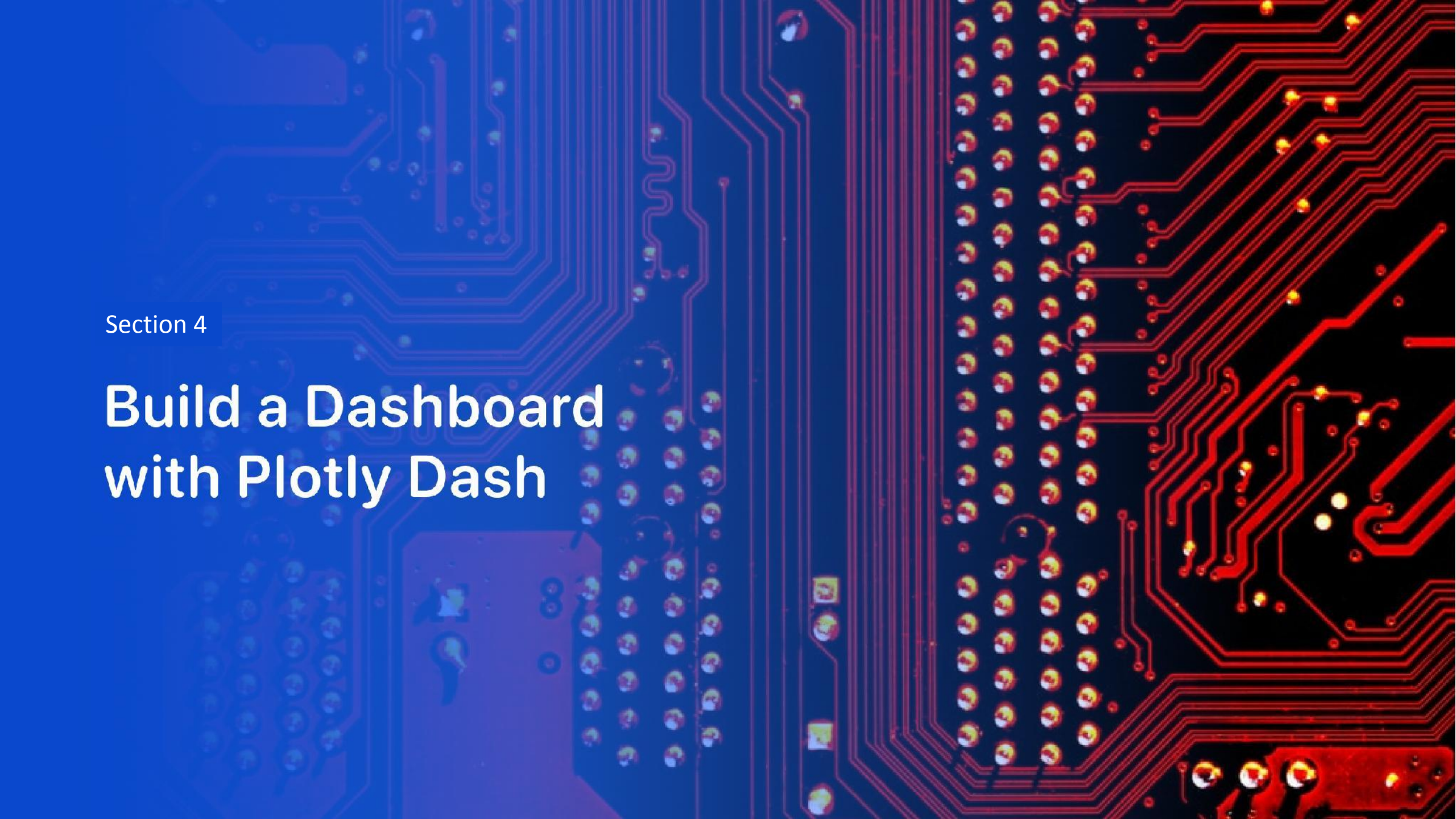
This is an example launch site (LC-40) with its successful (green) and failed (red) launches are marked with an appropriate tag.



Folium Line Distance

This is showcasing how folium can be used to draw a line showing distance from one point to another, in this case the distance from LC-40 to that point on the shoreline is calculated as 0.94 KM. This can be used for a variety of things.





Section 4

Build a Dashboard with Plotly Dash

Plotly Launch Records Pie Chart

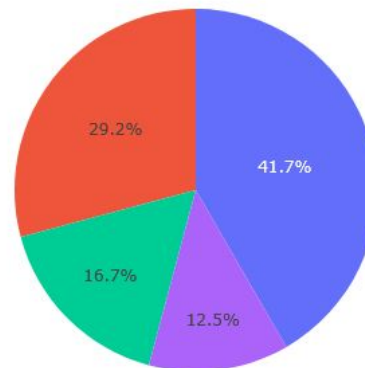
Below you can see the success rate for all launch sites in a pie chart. The most successful being KSC. It is also an interactable application so you can select a specific site and see its success rate.

SpaceX Launch Records Dashboard

All Sites

✕

Success Count for all launch sites



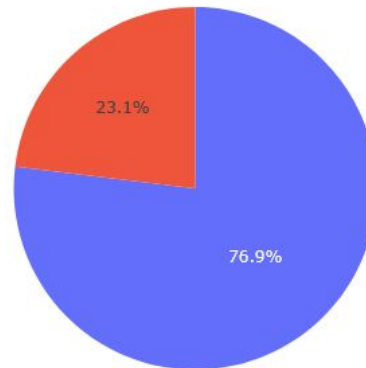
■ KSC LC-39A
■ CCAFS LC-40
■ VAFB SLC-4E
■ CCAFS SLC-40

The Most Successful Launch Site

KSC LC-39A has the most successful launch site with a 76.9% on the positive side and a 23.1 on the negative side. The key is on the right side with blue for success and orange for failure.

KSC LC-39A ✕ ▾

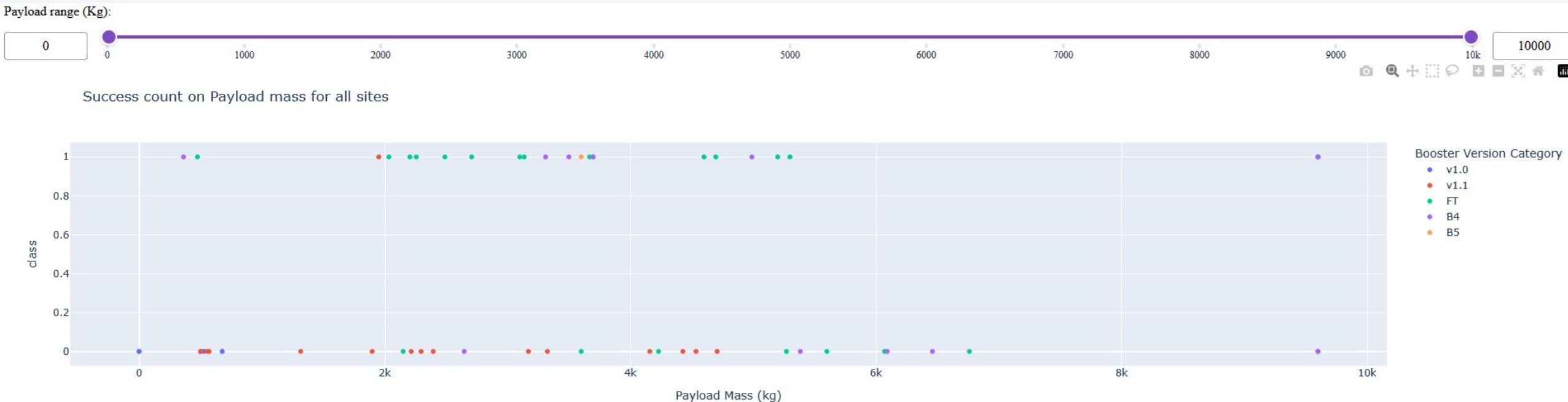
Total Success Launches for site KSC LC-39A



■ 1
■ 0

<Dashboard Screenshot 3>

Below you can see a scatterplot for successful and failed launches based on the booster version and Payload mass. This works in tandem with the pie chart selector, and you can limit sites based on your selection. The image below is pictured with all of them. You can limit the payload range to only show results between a specific number with the sliders/text boxes towards the top. You can also isolate the for specific booster versions by clicking others to make them disappear. B5 for instance has the highest success rate based on this chart as it only appears once and it was successful.



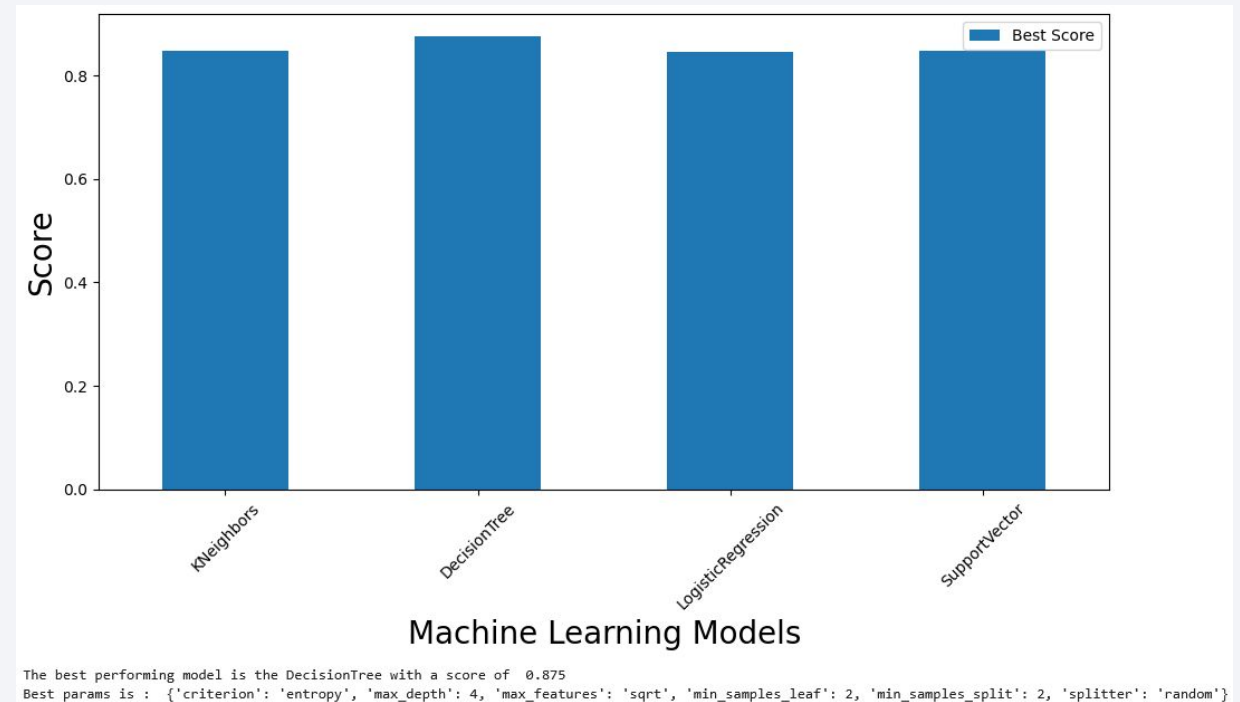


Section 5

Predictive Analysis (Classification)

Classification Accuracy

This is a bar chart made to showcase which ML model had the highest overall score. The result ended up being the decision tree with a score of 0.875.



Confusion Matrix

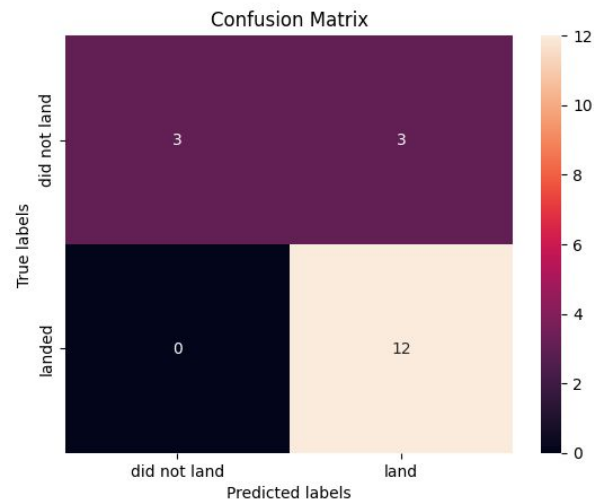
The confusion matrix for the decision tree classifier shows that the classifier can distinguish between the different classes. A major issue is the failed landings being marked as successful by the classifier.

```
print("Accuracy for decision tree classifier using the method score:", tree_cv.score(X_test, Y_test))
```

Accuracy for decision tree classifier using the method score: 0.8333333333333334

We can plot the confusion matrix

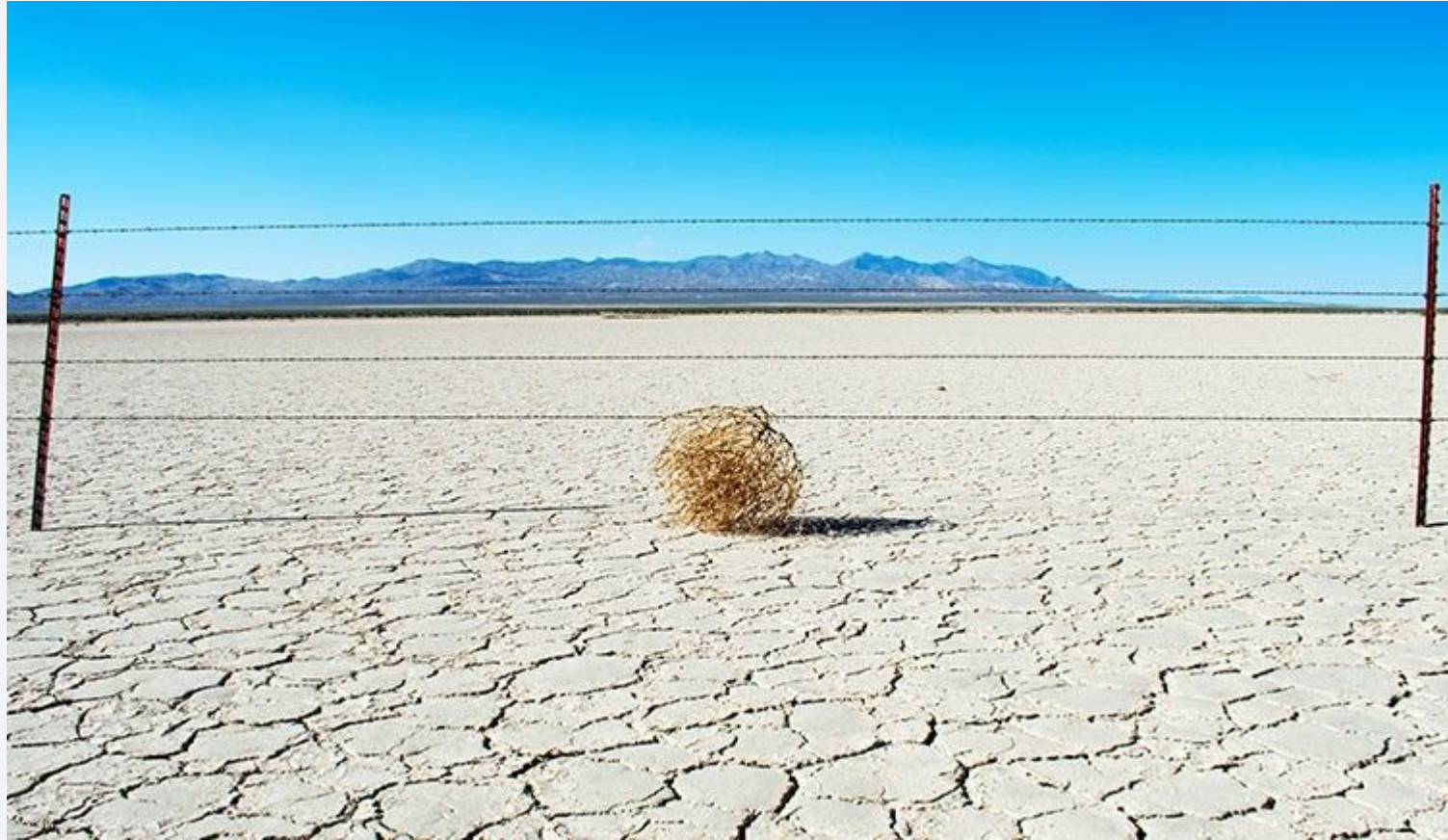
```
yhat = tree_cv.predict(X_test)  
plot_confusion_matrix(Y_test,yhat)
```



Conclusions

- Several facets can have an affect on a rocket launch.
- Based on this study, practice, and continually try, try, and trying again will increase your odds of success. This is evident in both the data and the timeline of successes from 2013-2019.
- The launch site of KSC LC 39A had the most successes.
- The orbits of ES-L1, GEO, HEO, SSO, VLEO had the highest success rates.
- The decision tree has proven to be the best ML algorithm for this task.

Appendix



Thank you!

